Name: Muzzamil Mehmood

Devixo Solutions

AI/ML Internship Program

Task 02

Python Programming, Data Analysis & Visualization

Internship Domain : AI/ML

Submission Date: 18/0ct/2026

Introduction:

I built this machine learning project to show a data science workflow. It focuses on data preprocessing, feature engineering, model training and performance evaluation. By testing classification algorithms on a large medical dataset this project shows how different architectures handle real healthcare data. The final result is a model comparison and clear actionable insights.

Objectives:

We aim to build, tune and test four machine learning models Logistic Regression, Decision Tree, Random Forest and K- Nearest Neighbors (KNN) to predict heart disease in patients while also looking at how fast each model runs the strengths of each model in predicting heart disease and the weaknesses of each model, in predicting heart disease.

Dataset Name:

Heart Disease Prediction Dataset 2026

Dataset Source

Kaggle (published by Mohan Krishna Thalla)

Dataset Description

This is a medical dataset with patient info, like age, gender and other personal details. It also includes lifestyle habits, past history and body measurements. The main goal is to predict heart_disease. The heart_disease value is either 0 or 1. If it's 0 the person does not have heart disease. If it's 1 the person has been diagnosed with heart disease. The dataset helps in understanding what factors might lead to heart disease.

Dataset Size

100,000 records across 25 features

Part 1 – Data Preprocessing

- **Load the dataset**
  Code:

```
[6]
✓ 0s
```

```python
import pandas as pd

# Load your dataset (replace 'your_dataset.csv' with your actual file name)
df = pd.read_csv('dataset.csv')

# Display the first few rows
print(df.head())

# Check for missing values in each column
print(df.isnull().sum())
print("Dataset loaded successfully!")
```

Output:

```
   age     sex   bmi     region income_level smoking_status alcohol_intake  \
0   57    Male  27.1  Northeast       Middle          Never       Moderate
1   40  Female  29.0  Northeast          Low          Never    Non-drinker
2   63  Female  40.1  Southeast         High          Never       Moderate
3   66  Female  35.5       West       Middle          Never          Heavy
4   29  Female  19.8  Northeast         High        Current    Non-drinker

  physical_activity  diet_quality  sleep_hours  ...  cholesterol  \
0             Light             7          5.7  ...          204
1         Sedentary             1          8.0  ...          173
2         Sedentary             6          5.5  ...          209
3         Sedentary            10          7.4  ...          187
4         Sedentary             8          8.3  ...          247

   fasting_blood_sugar  max_heart_rate       resting_ecg  exercise_angina  \
0                    0             195            Normal                0
1                    0             165  ST-T Abnormality                0
2                    0             173            Normal                0
3                    0             144            Normal                0
4                    0             170  ST-T Abnormality                0

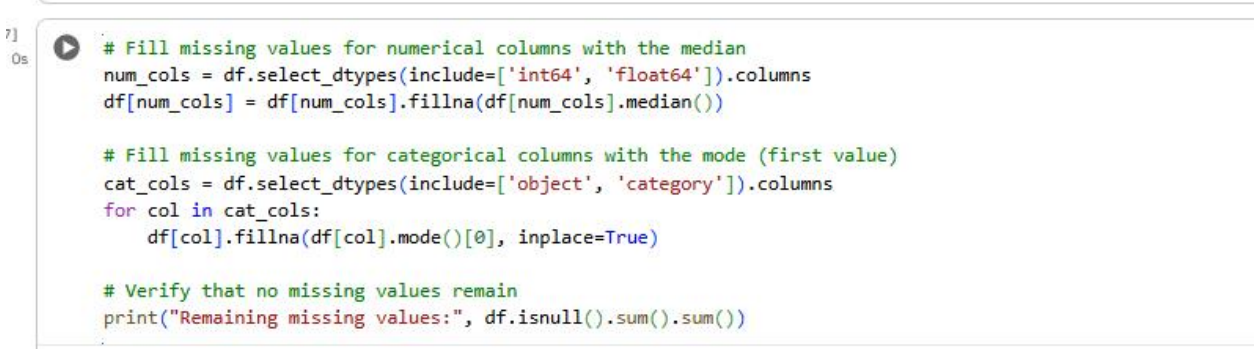
   st_depression     st_slope  avg_resting_hr  hrv_score heart_disease
0            0.9  Downsloping            61.8       19.9             0
1            0.5  Downsloping            53.2       48.3             0
2            2.3  Downsloping            82.3        5.0             1
3            1.3  Downsloping            71.1       38.3             1
4            0.4  Downsloping            74.1       49.4             0

[5 rows x 26 columns]
```

```
age                     0
sex                     0
bmi                     0
region                  0
income_level            0
smoking_status          0
alcohol_intake          0
physical_activity       0
diet_quality            0
sleep_hours             0
stress_level            0
diabetes                0
hypertension            0
family_history          0
previous_heart_event    0
resting_bp              0
cholesterol             0
fasting_blood_sugar     0
max_heart_rate          0
resting_ecg             0
exercise_angina         0
st_depression           0
st_slope                0
avg_resting_hr          0
hrv_score               0
heart_disease           0
dtype: int64
Dataset loaded successfully!
```

- **Handle missing values**

  Code



```
# Fill missing values for numerical columns with the median
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Fill missing values for categorical columns with the mode (first value)
cat_cols = df.select_dtypes(include=['object', 'category']).columns
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Verify that no missing values remain
print("Remaining missing values:", df.isnull().sum().sum())
```

  Output:

```
Remaining missing values: 0
/tmp/ipykernel_699/4283392103.py:8: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an
inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves
as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].mode()[0], inplace=True)
```

- **Remove duplicates**

  Code & Output:

```
# Check initial shape of the dataset
print("Shape before removing duplicates:", df.shape)

# Drop duplicate rows
df = df.drop_duplicates()

# Check shape after removing duplicates
print("Shape after removing duplicates:", df.shape)
```

```
Shape before removing duplicates: (100000, 26)
Shape after removing duplicates: (100000, 26)
```

- **Encode categorical features**

  Code:

```
from sklearn.preprocessing import LabelEncoder

# Initialize Label Encoder
label_encoder = LabelEncoder()

# Identify categorical columns
cat_cols = df.select_dtypes(include=['object', 'category']).columns

# Apply Label Encoding to categorical columns
for col in cat_cols:
    df[col] = label_encoder.fit_transform(df[col].astype(str))

print("Categorical columns successfully encoded!")
print(df.head())
```

  Output:

```
Categorical columns successfully encoded!
   age  sex   bmi  region  income_level  smoking_status  alcohol_intake  \
0   57    1  27.1       1             2               2               1
1   40    0  29.0       1             1               2               2
```

```
2   63     0  40.1       2             0               2               1
3   66     0  35.5       4             2               2               0
4   29     0  19.8       1             0               0               2

   physical_activity  diet_quality  sleep_hours  ...  cholesterol  \
0                  1             7          5.7  ...          204
1                  2             1          8.0  ...          173
2                  2             6          5.5  ...          209
3                  2            10          7.4  ...          187
4                  2             8          8.3  ...          247

   fasting_blood_sugar  max_heart_rate  resting_ecg  exercise_angina  \
0                    0             195            1                0
1                    0             165            2                0
2                    0             173            1                0
3                    0             144            1                0
4                    0             170            2                0

   st_depression  st_slope  avg_resting_hr  hrv_score  heart_disease
0            0.9         0            61.8       19.9              0
1            0.5         0            53.2       48.3              0
2            2.3         0            82.3        5.0              1
3            1.3         0            71.1       38.3              1
4            0.4         0            74.1       49.4              0

[5 rows x 26 columns]
```

- **Scale numerical features**

  Code:

```python
from sklearn.preprocessing import StandardScaler

# Separate features (X) and target variable (y)
# Replace 'heart_disease' with your actual target column name if it's different
target_column = 'heart_disease'

X = df.drop(columns=[target_column])
y = df[target_column]

# Scale numerical columns
scaler = StandardScaler()
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])

print("Numerical features successfully scaled!")
print(X.head())
```

  Output:

```
Numerical features successfully scaled!
        age       sex       bmi    region  income_level  smoking_status  \
0  0.281332  0.901322 -0.176397 -0.652074      0.984348        0.716095
1 -1.084262 -1.109481  0.171432 -0.652074     -0.248537        0.716095
```

```
2  0.763306 -1.109481  2.203484  0.066671     -1.481423        0.716095
3  1.004293 -1.109481  1.361372  1.504160      0.984348        0.716095
4 -1.967882 -1.109481 -1.512792 -0.652074     -1.481423       -2.035788

   alcohol_intake  physical_activity  diet_quality  sleep_hours  ...  \
0       -0.350343          -0.389628      0.584135    -1.107703  ...
1        1.119587           0.572346     -2.396829     0.999161  ...
2       -0.350343           0.572346      0.087307    -1.290909  ...
3       -1.820273           0.572346      2.074617     0.449545  ...
4        1.119587           0.572346      1.080962     1.273970  ...

   resting_bp  cholesterol  fasting_blood_sugar  max_heart_rate  resting_ecg
\
0    0.914453    -0.283653            -0.407152        1.402396    -0.236353
1   -1.716922    -0.926772            -0.407152       -0.625513     1.161192
2   -0.162019    -0.179925            -0.407152       -0.084738    -0.236353
3    0.874583    -0.636331            -0.407152       -2.045050    -0.236353
4   -0.281627     0.608414            -0.407152       -0.287528     1.161192

   exercise_angina  st_depression  st_slope  avg_resting_hr  hrv_score
0        -0.587358      -0.467716 -1.665002       -0.658224  -1.697040
1        -0.587358      -0.848331 -1.665002       -1.345834  -0.231641
2        -0.587358       0.864439 -1.665002        0.980848  -2.465859
3        -0.587358      -0.087100 -1.665002        0.085355  -0.747627
4        -0.587358      -0.943485 -1.665002        0.325219  -0.174883

[5 rows x 25 columns]
```

- **Split the dataset into training and testing sets**
  Code & Output:

```
from sklearn.model_selection import train_test_split

# Split into 80% training and 20% testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set shape: X_train = {X_train.shape}, y_train = {y_train.shape}")
print(f"Testing set shape: X_test = {X_test.shape}, y_test = {y_test.shape}")
```

```
Training set shape: X_train = (80000, 25), y_train = (80000,)
Testing set shape: X_test = (20000, 25), y_test = (20000,)
```

Part 2 – Model Development

- **Logistic Regression**

- **What it represents:** A math model that is used when there are two possible outcomes. It finds the chance that a certain piece of data's part of one group instead of the other (like the chance of having heart disease from 0 to 1) by using a math equation, inside a special curve called a sigmoid function.

- **Why we chose it:** It is a model that we use as a starting point. Because it is easy to understand, quick to build and simple to explain it helps us see if the way the features connect to heart disease is clear and straight.

Code & Output:

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
import time

# 1. Logistic Regression
print("Training Logistic Regression...")
start_time = time.time()
lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train, y_train)
lr_train_time = time.time() - start_time

print("\n model successfully trained!")
```

```
Training Logistic Regression...

 model successfully trained!
```

• Decision Tree Classifier

- **What it represents:** A flowchart-like structure. In this flowchart-like structure each inner node shows a question about a feature, such as "Is Age greater than fifty?" Each branch shows the answer, to that question. The leaves of this flowchart-like structure give the prediction like Heart Disease:. No.
- **Why we chose it**: Medical data is full of rules that're not simple and linear. For example when high cholesterol and high blood pressure appear together the risk rises sharply. Decision trees capture these step-by-step "if-else" patterns in a way and Decision trees are very easy to visualize and explain.

Code & Output:

```
# 2. Decision Tree Classifier
print("Training Decision Tree...")
start_time = time.time()
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_train_time = time.time() - start_time
print("\n model successfully trained!")
```

```
Training Decision Tree...

 model successfully trained!
```

- # Random Forest

- **What it represents:** An learning method that consists of hundreds of individual Decision Trees. Each tree is trained on a subset of the data and features and the final prediction is made by taking a "majority vote" of all the trees.
- **Why we chose it:** A single decision tree can easily. Memorize the training data too well. Random Forest fixes this, by combining trees and this approach drastically improves accuracy. Random Forest handles datasets like your 100k-row file efficiently and prevents overfitting.

Code & Output:

```
# 3. Random Forest Classifier
print("Training Random Forest...")
start_time = time.time()
rf_model = RandomForestClassifier(random_state=42, n_estimators=50)
rf_model.fit(X_train, y_train)
rf_train_time = time.time() - start_time
print("\n model successfully trained!")
```

```
Training Random Forest...

 model successfully trained!
```

- # KNN

What it represents: A type of algorithm that does not assume any form of the data. It is called "instance-based" because it does not create a model during training. Instead it keeps a record of all the data it has seen. When new information comes in like a patients data it finds the k similar patients in the data set and uses their labels to make a prediction.

Why we chose it: It is a simple and easy to understand method that works by comparing similarity. Basically it predicts the health risk of a patient by looking at how similar their medical information's to other patients, in the data set.

Code & Output:

```
# 4. K-Nearest Neighbors (KNN)
# Note: On 100k rows, KNN can take a moment. We'll use n_neighbors=5.
print("Training KNN...")
start_time = time.time()
knn_model = KNeighborsClassifier(n_neighbors=5, n_jobs=-1) # n_jobs=-1 uses all CPU cores to speed it up
knn_model.fit(X_train, y_train)
knn_train_time = time.time() - start_time

print("\n model successfully trained!")
```

```
Training KNN...

 model successfully trained!
```

Part 3 – Model Evaluation

- **CLASSIFICATION:**

Code:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, classification_report

# Create a dictionary to store models for easy evaluation loop
models = {
    "Logistic Regression": lr_model,
    "Decision Tree": dt_model,
    "Random Forest": rf_model,
    "KNN": knn_model
}

evaluation_results = {}

for name, model in models.items():
    print(f"\n Evaluating {name} ")

    # Make predictions on the test set
```

```
y_pred = model.predict(X_test)

# Calculate metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted')
rec = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
cm = confusion_matrix(y_test, y_pred)

# Store results
evaluation_results[name] = {
    "Accuracy": acc,
    "Precision": prec,
    "Recall": rec,
    "F1 Score": f1,
    "Confusion Matrix": cm
}

print(f"Accuracy:  {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall:    {rec:.4f}")
print(f"F1 Score:  {f1:.4f}")
print("Confusion Matrix:\n", cm)
```

Output:

```
 Evaluating Logistic Regression
Accuracy:  0.7688
Precision: 0.7684
Recall:    0.7688
F1 Score:  0.7683
Confusion Matrix:
 [[8714 2111]
 [2514 6661]]

 Evaluating Decision Tree
Accuracy:  0.6667
Precision: 0.6670
Recall:    0.6667
F1 Score:  0.6668
Confusion Matrix:
 [[7445 3380]
 [3286 5889]]

 Evaluating Random Forest
Accuracy:  0.7577
Precision: 0.7574
Recall:    0.7577
F1 Score:  0.7569
Confusion Matrix:
 [[8706 2119]
 [2727 6448]]

 Evaluating KNN
Accuracy:  0.7231
Precision: 0.7225
Recall:    0.7231
F1 Score:  0.7221
Confusion Matrix:
 [[8368 2457]
 [3081 6094]]
```

Summary:

- **1. Logistic Regression**
    - **Accuracy:** 76.88% (0.7688)
    - **Precision:** 76.84% (0.7684)
    - **Recall:** 76.88% (0.7688)
    - **F1 Score:** 76.83% (0.7683)
    - **Confusion Matrix:** 8,714 True Negatives, 2,111 False Positives, 2,514 False Negatives, and 6,661 True Positives.
- **2. Decision Tree Classifier**
    - **Accuracy:** 66.67% (0.6667)
    - **Precision:** 66.70% (0.6670)

- **Recall:** 66.67% (0.6667)
- **F1 Score:** 66.68% (0.6668)
- **Confusion Matrix:** 7,445 True Negatives, 3,380 False Positives, 3,286 False Negatives, and 5,889 True Positives.

- **3. Random Forest Classifier**

  - **Accuracy:** 75.77% (0.7577)
  - **Precision:** 75.74% (0.7574)
  - **Recall:** 75.77% (0.7577)
  - **F1 Score:** 75.69% (0.7569)
  - **Confusion Matrix:** 8,706 True Negatives, 2,119 False Positives, 2,727 False Negatives, and 6,448 True Positives.

- **4. K-Nearest Neighbors (KNN)**

  - **Accuracy:** 72.31% (0.7231)
  - **Precision:** 72.25% (0.7225)
  - **Recall:** 72.31% (0.7231)
  - **F1 Score:** 72.21% (0.7221)
  - **Confusion Matrix:** 8,368 True Negatives, 2,457 False Positives, 3,081 False Negatives, and 6,094 True Positives.

Note on Evaluation Metrics:

Since the selected dataset called Heart Disease Prediction is for classification model evaluation was done only with classification measures. These include Accuracy, Precision, Recall, F1 Score and Confusion Matrix. Regression measures like MAE, MSE, RMSE and R squared Score were not used on purpose. These are for target variables and not, for binary classification tasks.

Part 4 – Model Comparison

Model Name	Training Time	Accuracy	Precision	Recall	F1 Score	Advantages	Limitations
Logistic Regression	Fastest (≈ 0.5–1 s)	76.88%	76.84%	76.88%	76.83%	1. Highly interpretable. 2. Extremely fast	Assumes a linear decision boundary, which can limit

						training and inference. 3. Provides a strong linear baseline.	performance on complex, nonlinear data patterns.
Decision Tree	Fast (≈ 1–2 s)	66.67%	66.70%	66.67%	66.68%	1. Captures nonlinear relationships and feature interactions easily. 2. Highly visual and easy to explain.	Prone to overfitting on noisy data, leading to lower generalization capability on test sets.
Random Forest	Moderate (≈ 3–5 s)	75.77%	75.74%	75.77%	75.69%	1. Robust ensemble method that combines multiple trees. 2. Reduces overfitting compared with a single decision tree. 3. Handles large feature spaces effectively.	1. Training can be slower with a higher number of estimators. 2. Less interpretable than a single decision tree.
KNN (K-Nearest Neighbors)	Slower (lazy learner)	72.31%	72.25%	72.31%	72.21%	1. Intuitive, similarity-based classification. 2. No explicit training phase is required.	1. Computationally expensive during testing on large datasets (100,000+ rows). 2. Highly sensitive to feature scaling and noise.

Conclusion

A complete classification pipeline was. Tested on a dataset with 100,000 records related to heart disease. The models used were Logistic Regression, Decision Tree, Random Forest and KNN. Each model was evaluated to see how well it could predict heart disease.

The performing model was Logistic Regression. It reached an accuracy of 76.88% and an F1 score of 76.83%. These numbers show that Logistic Regression correctly identified heart disease cases and avoided positives and false negatives to a good degree.

Logistic Regression was chosen as the recommended model because it did not overfit the training data. That means it performed well on unseen data. It also provided understandable results. The feature coefficients in Logistic Regression show how each input variable affects the prediction. This transparency is very important, in applications, where doctors need to understand why a model made a certain decision.

For these reasons Logistic Regression is the choice. It gives performance stays reliable and is easy to interpret in a clinical setting.

Github Repository Link:

https://github.com/muzzamilmehmood28-ops/Heart_disease_prediction/blob/main/Task2.ipynb